Agent 長期記憶系統

框架對比、認知架構與選型決策

報告日期：2026-04-29

撰寫方：hv-research / 2026-04-29

目錄

1. **agent-memory-systems_橫縱分析報告**

Agent 長期記憶系統：框架對比、認知架構與選型決策

> 橫縱分析法深度研究報告 ｜ 2026-04-29 ｜ 資料截止：2026-04-29

§0 摘要：60 秒讀完

2026 年是 agent 元年的喧囂背後，最大的工程難題不是 reasoning、不是 tool use、是**記憶**。Karpathy 把這個現象命名為「**Decade of Agents**」——當前 LLM 像得了 anterograde amnesia 的天才，每次對話結束就忘光，所有持續性都靠工程外掛硬撐。

這份報告盤點當前主流的解法，把它們攤在兩個維度上：

```
                Hierarchical / Graph
                        ▲
                        |
          Letta •       |     • Zep / Graphiti
        (OS paging)     |   (Temporal KG)
                        |
                        |     • Karpathy LLM Wiki
                        |     (Compiled markdown)
 ———————————————————————┼———————————————————————►
                        |              Extraction
    Verbatim            |
                        |     • Mem0
          MemPalace•    |     (LLM extraction + hybrid store)
        (Method of      |
          Loci)         |
                        ▼
                   Flat store
```

五大流派的核心取捨：

框架	核心隱喻	Write 成本	Retrieval	LongMemEval	何時用
Mem0	LLM extraction + hybrid	高（每次 LLM call）	快	49.0%	個人化 chatbot / B2B copilot
Zep / Graphiti	Temporal Knowledge Graph	中（自動 KG）	快、無 LLM	**63.8%**	事實會變的長期 agent
Letta（前 MemGPT）	OS paging（RAM/Disk）	中	LLM 自決	無公開	長 horizon autonomous
MemPalace	Method of Loci 空間隱喻	**零 LLM**	階層喚醒	96.6%（爭議大）/ 88.9%（獨立重現）	離線 / 隱私敏感
Karpathy LLM Wiki	Compiler（編譯一次）	中（人 + LLM 協作）	markdown 直查	無 benchmark	個人 PKM / 開發者 second brain

最值得深思的不是哪家數字最高，而是 **Karpathy 在這場戰局裡丟下的三顆思想炸彈**：

1. **System Prompt Learning** ——LLM 缺一個學習範式，不是 pretraining 也不是 fine-tune，而是「會自己寫筆記、總結教訓」
2. **LLM Wiki** ——這個範式的具體實作：對話 → daily log → 編譯成 wiki → 注入下次 session
3. **Cognitive Core** ——最終形態：瘦核心模型 + 外掛記憶，「I want to remove the memory... and only maintain the algorithms for thought」

如果 Karpathy 對了，現在這五家全是過渡形態。**真正的記憶最終要烤回權重**——這是 § 5.4 + § 9.9 的重點。

---

§ 1 為什麼 agent 需要「真正的記憶」

§ 1.1 Anterograde Amnesia——Karpathy 的隱喻

順向失憶症（anterograde amnesia）是一種神經學病症：患者保有過去的記憶，但無法形成新的長期記憶。每天醒來，昨天發生的事都歸零。1985 年的英國音樂家 Clive Wearing 是最有名的個案，他每隔幾分鐘就「醒過來」一次，每次都以為自己剛從昏迷中復甦，認得出妻子但記不住她下午來看過。

Karpathy 用這個病作為當前 LLM 的隱喻。他在 2025 年 10 月接受 Dwarkesh Patel 訪談時說了一段被廣泛轉貼的話：

> "They don't have continual learning. You can't just tell them something and they'll remember it."

訓練結束後，模型權重就凍結了。你在第 N 次對話告訴它「我喜歡用咖哩粉而不是咖哩塊」，它在 N+1 次對話完全沒印象。它擁有 175B 參數的「過去記憶」（pretraining 時看過的世界），但**沒有形成新長期記憶的能力**。

短期記憶呢？有，那叫 context window。但 context window 是個房間，不是腦——你關門離開，房間就空了。

§ 1.2 Decade of Agents——四大缺口

Karpathy 把當前 agent 的問題列為「2026 年是 agent 元年」這種樂觀派的反駁。他說這應該叫 **Decade of Agents**——要十年才能讓 agent 達到「員工 / 實習生水準的可靠性」。

四大缺口（前三項是他明確列出的，第四項記憶是他多次強調的隱含項）：

缺口	現狀
Multimodality	圖像、語音、影片整合仍邊緣
Computer use	瀏覽器、檔案、IDE 操作不可靠
Continual learning（記憶）	訓練後無法 consolidate 新知識
Cognitive capacity	"They're cognitively lacking and it's just not working."

> "It will take about a decade to work through all of those issues."

四項裡最尖銳的是記憶。其他三項是能力延伸，記憶是**架構性缺陷**——沒有持續性，所有「員工級」的承諾都不成立。一個記不住昨天交辦事項的實習生，沒有公司會要。

§1.3 三種解法的覆蓋範圍

業界當前的三條路在 cover 不同的時間跨度：

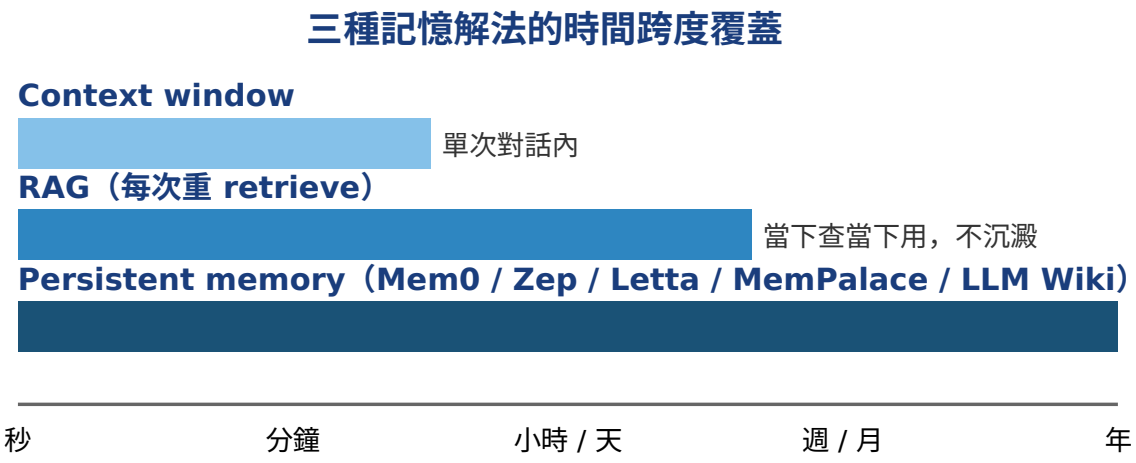

圖 1：三種記憶解法的時間跨度覆蓋——context window 只能撐單次對話；RAG 在 query 時臨時拉資料，不累積；persistent memory 才真正跨 session。

Context window 即使做到 1M tokens，仍然是「房間」不是「腦」——關門就空。RAG 是個臨時資料庫，每次重新 query 都從零開始 synthesis。**只有 persistent memory 那條長條真正回答了 Karpathy 的問題**：跨 session、跨月、跨年的連續性。

但 persistent memory 的實作差異巨大。下一章把這條長條切開來看。

---

§2 五大流派：橫向對比

§2.0 速查表

進深度對比前先給一張全景：

框架	創立 / 釋出	開源	核心隱喻	主場景	GitHub stars
Mem0	2024 中	✓（Pro 鎖 graph）	Hybrid store（vector + graph + KV）	個人化 chatbot	~48K
Zep / Graphiti	2024 末	✓（Graphiti 開源）	Temporal Knowledge Graph	事實演變的長期 agent	~10K
Letta（前 MemGPT）	2023.10 paper / 2024 商業化	✓	OS paging（RAM/Disk）	長 horizon autonomous	~15K
MemPalace	2026-04-08	✓（MIT）	Method of Loci 空間	離線 / 隱私	22K → 47K（兩週爆紅）
Karpathy LLM Wiki	2026-04-04（gist）	概念	Compiler（編譯一次）	個人 PKM	概念 gist

橫軸把它們排到 verbatim ↔ extraction、flat ↔ hierarchical 兩個維度，分布像本報告 §0 那張小圖：MemPalace 在左下（verbatim + flat-ish）、Letta 在左上（OS-style hierarchical）、Zep 在右上（extraction-heavy graph）、Mem0 在右下偏中（extraction + hybrid）、LLM Wiki 在右上（compiled markdown 是 hierarchical 的一種變體）。

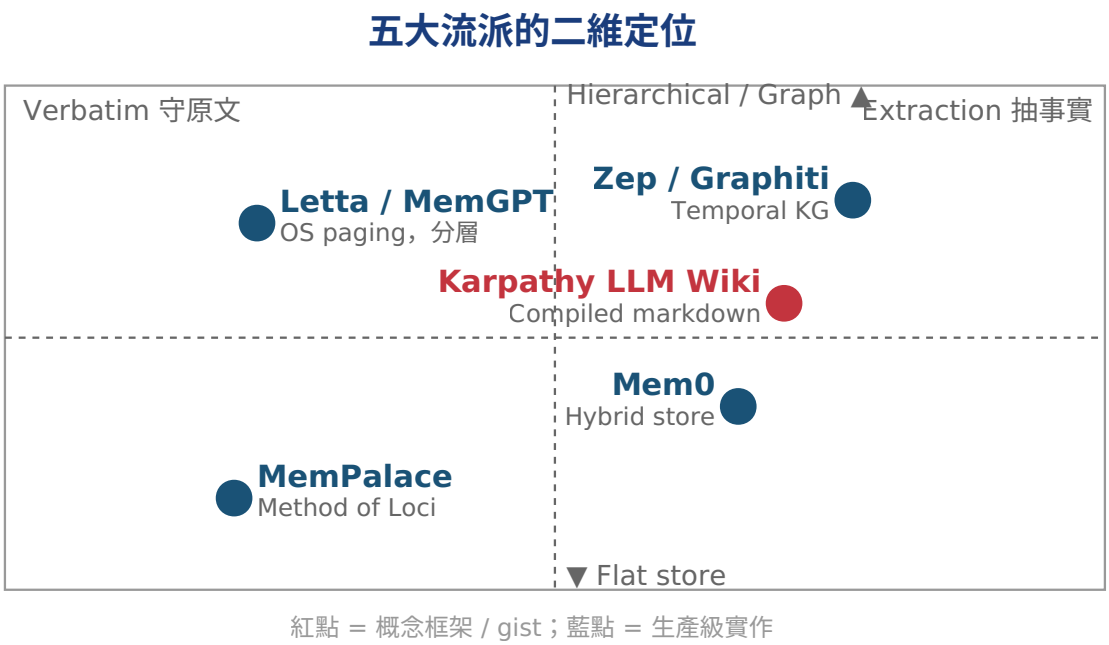

圖 2：五大流派在「儲存哲學 × 結構化程度」二維平面的定位。

§2.1 Mem0 ——社群最大的 hybrid store

Mem0 出生於 2024 年中，是當前社群最大的獨立記憶框架（~48K stars）。它的定位是「drop-in memory layer」——你在原本 stateless 的 agent 外面包一層 service，不需動 agent 邏輯。

架構：

- **三類記憶**：episodic（what happened）/ semantic（what is known）/ procedural（how to do）
- **Multi-scope**：`user_id`、`agent_id`、`run_id/session_id`、`app_id/org_id`，retrieval 時 compose
- **Hybrid storage**：vector store（19 個 backend）+ graph variant **Mem0g**（Kuzu / Neo4j）+ KV
- **三種 hosting**：managed cloud / open-source self-hosted / local MCP

Write path：每次對話結束後 LLM 抽取「這段對話有什麼值得記」→ 寫入 vector / graph。
這是 Mem0 的雙面刃——精準但每次都要付 LLM extraction 成本。

Mem0 自家 LOCOMO benchmark（ECAI 2025，arXiv:2504.19413）：

方法	LLM 分數	P95 Latency	Tokens/conv
Full-context	72.9%	17.12s	~26,000
Mem0g (graph)	68.4%	2.59s	~1,800
Mem0 (vector)	66.9%	1.44s	~1,800
RAG	61.0%	0.70s	—
OpenAI Memory	52.9%	—	—

讀法：Mem0 沒贏 full-context，但在 92% latency 削減 + 93% token 削減的條件下逼近 92% 的準度。**這個 trade-off 才是 Mem0 的真實賣點**——「我幾乎跟全塞 context window 一樣準，但便宜 10 倍」。

弱點（Atlan 2026 整理）：
- Graph memory 鎖在 Pro tier
- 無 temporal fact modeling（fact 變了就直接覆蓋，不留歷史）
- **LongMemEval 49.0%（GPT-4o）**——比 Zep 低 15 點

社群最大不等於技術最強。Mem0 贏在生態整合（13 個 agent framework）、Voice agent（ElevenLabs / LiveKit）、SOC 2 Type II 合規，輸在 fact 演變的細膩處理。

§2.2 Zep / Graphiti ——時序知識圖譜

如果你的 agent 需要記住「使用者三個月前說喜歡 cold brew，但上週改說 latte」，Zep 是當前最強的選擇。它的核心隱喻不是 store，是 **temporal knowledge graph**。

Bi-Temporal Model——每個 fact 帶兩種時間戳：

- Event time：這件事**何時發生**
- Transaction time：這件事**何時被記錄 / 失效**

> "facts have validity windows. When information changes, old facts are invalidated — not deleted."

舊 fact 不刪除，只標記 invalidated。你可以查「現在使用者偏好什麼」（current state），也可以查「三個月前他說什麼」（historical state）。這正是 Karpathy 在 §5.4 會講到的 episodic memory 的雛形。

儲存模式：
- 三元組（Entity → Relationship → Entity）+ validity window
- 自動從非結構化資料抽取
- 「Everything traces back to episodes — the raw data that produced it」（保 provenance）

Graphiti（開源核心） vs Zep platform（managed）：

層	角色	適合誰
Graphiti	開源 temporal context graph engine	自建偏好者
Zep	managed infra：governance + threads + 預配 retrieval + dashboards	企業 turnkey

性能：query 時不需 LLM call（fact 已抽好），P95 retrieval ~300ms，Atlan 評為「事實演變類場景的最強選擇」。

LongMemEval 63.8%（GPT-4o）——本對照表中最高的一個生產級數字。
口徑警告：是 Zep 自己跑的 GPT-4o 配置，跟 Mem0 那 49.0% 同口徑可比，但跟 MemPalace 那 96.6% R@5 完全是不同 metric（前者測答對率，後者測 retrieval recall）。§4 會專門拆這個。

弱點：社群比 Mem0 小、enterprise 訂價不透明、無 constitutional governance layer（合規仍要自己加）。

§2.3 Letta（前 MemGPT）——agent OS

2023 年 10 月 arXiv:2310.08560 發表的 MemGPT paper 是這場記憶之戰的學術起點。它丟出一個讓人眼睛一亮的隱喻：**LLM 的 context window 像 RAM，外部儲存像 disk，LLM 自己當作業系統 + page in/out**。

> "inspiration from hierarchical memory systems in traditional operating systems that provide the appearance of large memory resources through data movement between fast and slow memory"

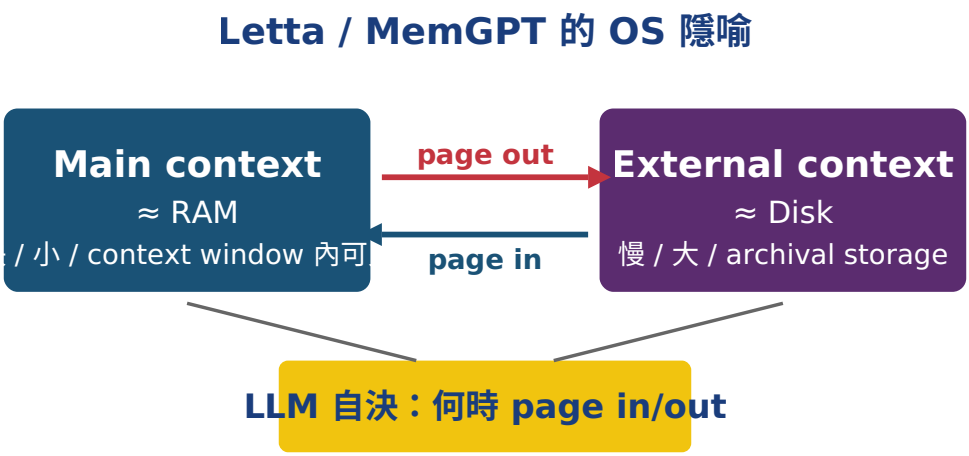

圖 3：Letta / MemGPT 把 LLM 當作業系統，main context 對應 RAM、external context 對應 disk，由 LLM 自己控制 page in / out。

核心差異（vs Mem0 / Zep）：
- Mem0 / Zep 把 agent 當 stateless function，外面包 memory service
- **Letta 把 agent 當 persistent entity**，自己管 state、自己決定何時 archive、何時 retrieve

Memory Block 概念：
- 帶標籤的 memory blocks（如 "human"、"persona"）
- Agent 可在互動中 reference 並 update 各 block
- 支援多 agent 共享 memory blocks（為第二題編排研究預留）

商業化版本（2024 後 rebrand）：

產品	對標	場景
Letta Code SDK	Claude Code、Codex	本地 tool execution + skills
Letta API	OpenAI Responses API	純 chatbot
MemFS	（新功能）	git-tracked memory

弱點：非 drop-in，需採完整 agent runtime；社群比 Mem0 小；訂價不透明；無公開 LongMemEval 數字。

為什麼仍重要：對「需要連續跑數天、跨任務累積技能」的 autonomous agent，Letta 是目前唯一原生支援的。Mem0 / Zep 都假設 agent 跑一次就停，Letta 假設 agent 永遠在跑。

§2.4 MemPalace ——4 月份的爆紅與爭議

2026-04-08 釋出，兩週內 GitHub 47K stars，宣稱在 LongMemEval 上達到 96.6% R@5——比 Zep 的 63.8% 高出 33 點。社群一開始震撼，幾天後批評文鋪天蓋地。

核心架構：四層記憶堆疊

Layer	Tokens	載入時機	內容
L0 Identity	~50	啟動時必載	AI persona
L1 Critical Facts	~120（AAAK 格式）	啟動時必載	team / projects / preferences
L2 Room Recall	按需	對話觸發	近期 session 重點
L3 Deep Search	按需	semantic search	全部 compressed memory vaults

170-token 喚醒成本——啟動只需 L0 + L1。對比：raw history 19.5M tokens、LLM-summarized 約 650K。年成本從 ~$507 降到 ~$0.70（純 init）。

空間隱喻（method of loci）——這是它命名的根：

- Wing：major categories
- Room：sub-topics
- Hall：shared corridors（facts / events / discoveries / preferences / advice）
- Closet：compressed summaries（AAAK 格式）
- **Drawer：verbatim originals（不丟原文）**
- Tunnel：cross-wing connections

雙層儲存哲學：closets 存壓縮 summary，drawers 存原檔。原文可恢復，不像 extraction 派 compress 後就丟。**這是它真正的差異化**——不是空間隱喻本身，是 verbatim-first。

96.6% 爭議（Nicholas Rhodes 批評文，2026-04-08，11 日更新）：

四點方法論問題：

1. **Hand-tuning**：找出 failing questions 後針對性修補再重測（違反 benchmark integrity）
2. **Retrieval window gaming**：top_k=50 但 dataset 只有 19-32 items → 「memory system 貢獻為零」
3. **Metric confusion**：分數量的是 retrieval presence 而非 answer correctness
4. **API dependency**：100% 配置需要付費 Claude API，但 marketing 強調 local-only

獨立重現：88.9% R@10（hybrid retrieval，無 LLM）；無 reranking 時掉到 60.3%。

> "ChromaDB is doing the heavy lifting, not MemPalace."

批評者也承認**真正的創新點**：
- Verbatim 儲存哲學
- 階層架構（雖然空間隱喻本身貢獻有限）
- MIT 授權、純 open source、純 local
- MCP 整合 19 個工具

真正的客群——批評者點得很準：「ideal for infrastructure-owning developers, not for operators seeking managed cross-tool memory solutions」。

§2.5 Karpathy LLM Wiki ——編譯器路線

2026-04-04，Karpathy 在 GitHub 上發了一個 1500 字的 gist（karpathy/llm-wiki）。沒有程式碼、沒有 benchmark、沒有 demo——就是一個架構主張。但它在 4 月份引爆的討論量超過 MemPalace、超過 Mem0 的 1.0.4 release，**因為它在問一個更根本的問題：當前所有記憶系統的範式對不對？**

三層架構：

1. **Raw sources（不可變）**：策展文件——文章、論文、圖、資料檔
2. **The wiki（LLM 維護）**：markdown 檔——summaries / entity pages / concept pages / cross-references
3. **The schema（配置）**：一個文件（如 `CLAUDE.md`）定義 wiki 結構、慣例、workflow

核心模式：incremental，不是 retrieval-only。每個新 source 觸發：

- 閱讀 + 討論
- 寫 summary 頁
- 更新 index
- 修訂相關 entity / concept 頁面（每個 source 約 10-15 頁）
- append log entry

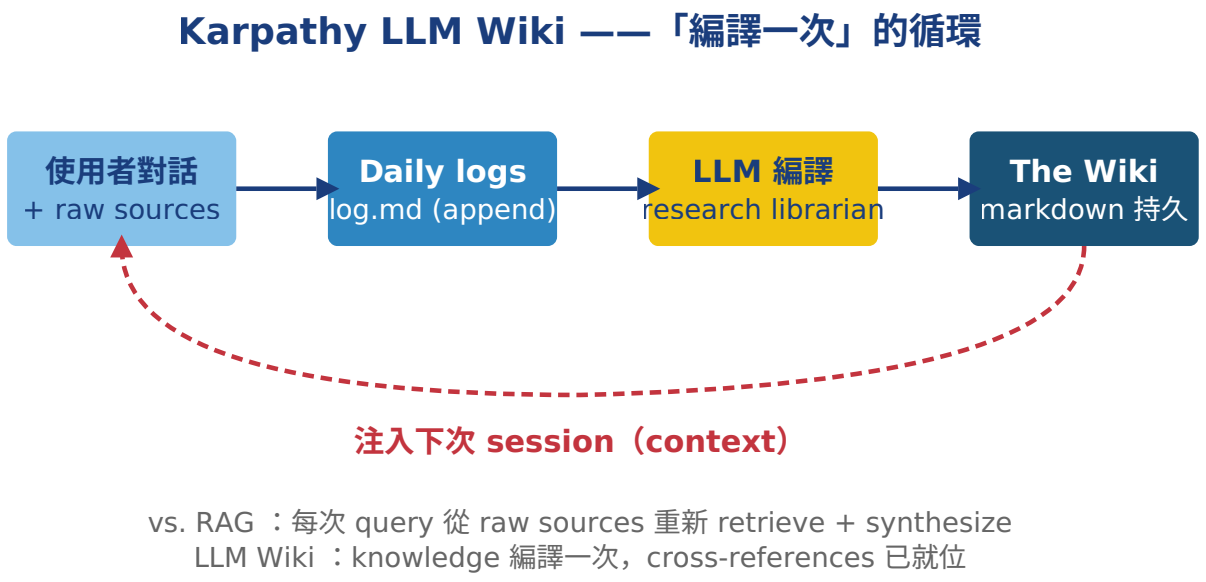

圖 4：Karpathy LLM Wiki 的循環——對話 → daily log → LLM 編譯 → wiki → 注入下次 session。核心差異是「knowledge 編譯一次」而非「每次重新 synthesize」。

最關鍵的對 RAG 差異：

> "Most people's experience with LLMs and documents looks like RAG: you upload a collection of files, the LLM retrieves relevant chunks at query time, and generates an answer. This works, but **the LLM is rediscovering knowledge from scratch on every question**."

vs.

> "the LLM **incrementally builds and maintains a persistent wiki**"

LLM 的角色：research librarian——不是分析師，是檔案管理員：

> "You're in charge of sourcing, exploration, and asking the right questions. The LLM does all the grunt work — the summarizing, cross-referencing, filing, and bookkeeping that makes a knowledge base actually useful over time."
>
> "Obsidian is the IDE; the LLM is the programmer; the wiki is the codebase."

社群衍生：

- **rohitg00 LLM Wiki v2** ——加 citation validator、rotation policy、conflict resolution
- **Memco / Gamgee / Epsilla** ——商業化嘗試
- **DEV.to「I Over-Engineered Karpathy's Agent Memory」** ——批評實作落地

Karpathy 自承的限制：

- "This document is intentionally abstract. It describes the idea, not a specific implementation."
- index.md 在 ~100 sources / 數百頁規模 work surprisingly well，更大規模需 qmd 或自製搜尋
- **未明確處理 LLM hallucination / citation grounding**

最重要的本質區別：Mem0 / Zep / Letta / MemPalace 都在做「**怎麼把記憶塞進 query path**」；Karpathy 在做「**怎麼讓 LLM 自己整理記憶**」。前者是 retrieval engineering，後者是 knowledge curation——這是後面 §3.3「Beyond RAG」之爭的根。

§2.6 次主流框架（一覽表）

主菜以外，2026 年活躍的次主流框架：

框架	一句話定位	GitHub	何時考慮
LangChain / LangMem	LangChain 生態的 modular memory	~100K（LangChain）	已在 LangGraph 上
Cognee	Local-first poly-store graph，純離線	~7K	Air-gapped、GDPR 嚴格
Supermemory	MCP-native，整合 Claude Code	—	Coding agent 跨 session
SuperLocalMemory	Local-only 變體	—	個人 PoC
MemoClaw	輕量 wrapper	—	快速原型
ByteRover	LLM-curated hierarchical context	—	學術近實驗

值得注意：**Supermemory 自報跨 LongMemEval / LoCoMo / ConvoMem 全領先**，但無第三方驗證——這在 2026 年的 benchmark inflation 浪潮裡屬於「先信一半」的情況。

§2.7 學術原型（ICLR 2026 MemAgents Workshop）

ICLR 2026 收的 MemAgents workshop 是當前學術前沿匯總：

系統	核心貢獻
HiMem	Hierarchical Long-Term Memory for Long-Horizon Agents
SYNAPSE	Episodic-Semantic Memory via Spreading Activation（人腦聯想啟發）
TiMem	Temporal-Hierarchical Memory Consolidation
MAGMA	Multi-Graph based Agentic Memory Architecture
GAM	Graph-based Agentic Memory（Semantic-Event-Triggered，episodic→semantic consolidation）
A-MEM	Zettelkasten-inspired，dynamic linking
MIRIX	六模組（Core / Episodic / Semantic / Procedural / Resource / Knowledge Vault）
Coarse-to-Fine	多層 memory 支援 exploration + 錯誤修正

emergentmind 整理出 recall 數學模型：

$$ p(t) = 1 - \exp(-r \cdot e^{-a t}) $$

組合 relevance r、time decay a、頻率，給 recall probability。**這個模型是當前生產級框架幾乎都沒實作的**——大家都靠 vector similarity 直接做，而不是建模 forgetting curve。學術前沿與生產級的 gap 仍很大。

---

§3 設計取捨深度剖析

§3.1 Write-time vs Read-time 成本

這是所有記憶框架的第一個分岔。Write-time 成本指**寫入記憶時的代價**（通常是 LLM call、CPU、儲存）；read-time 成本指**查詢時的代價**（latency、token）。

框架	Write 成本	Write 需 LLM	Read latency	Read 需 LLM
Mem0	高	✓（每次 extraction）	1.44s	✗
Zep / Graphiti	中	✓（KG 抽取）	~300ms	✗
Letta	中	✓（agent 自決 archive）	變動	✓（page in 時）
MemPalace	**零**	✗（verbatim 直存）	變動（L0/L1 即時）	✗
LLM Wiki	中	✓（research librarian 編譯）	變動（markdown 直查）	部分

兩個極端：
- **Mem0 / Zep**：寫入很貴（LLM 抽事實 / 三元組），但寫完之後 read 飛快、不需 LLM
- **MemPalace**：完全不付 write LLM 費用，但 read 時要靠 retrieval engine（ChromaDB）+ 階層走訪

LLM Wiki 是個第三條路：write 時付 LLM 編譯費，但**這是攤提的**——一次編譯，N 次 read 受惠。對 personal 場景特別划算（一個人一天對話量小，編譯成本可接受）。

§ 3.2 Verbatim vs Extraction vs Compiled-Wiki ——三派哲學

如果把「資訊進入記憶系統時被怎麼處理」當作維度，2026 年清楚分成三派。

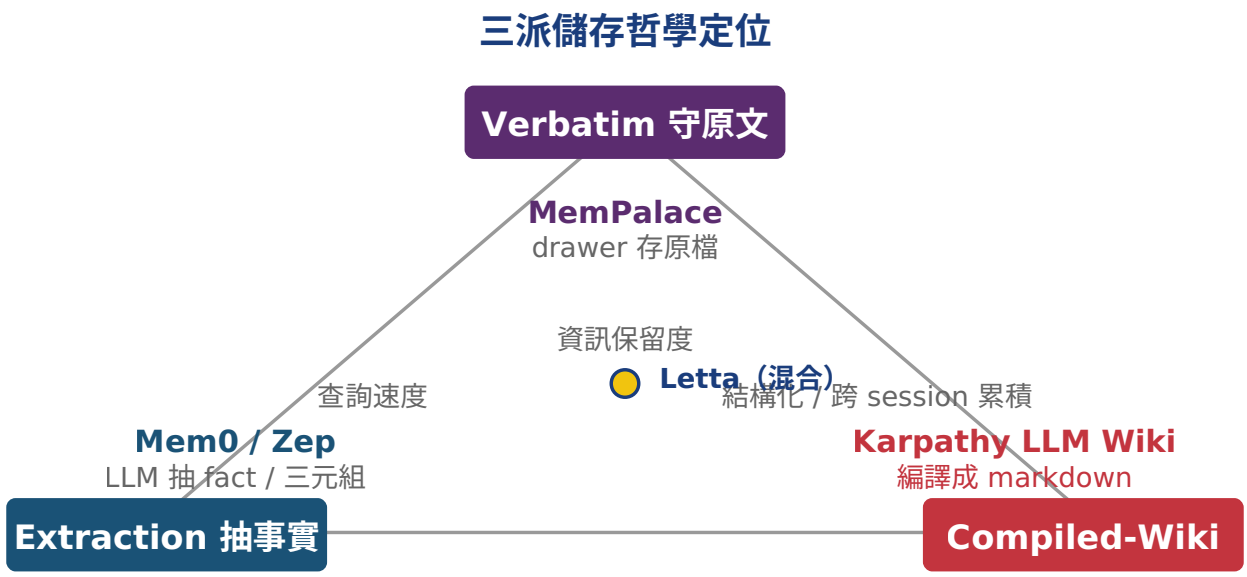

圖 5：三派哲學的三角定位——Verbatim（守原文）、Extraction（抽事實）、Compiled-Wiki（編譯成結構化文件）。Letta 因為 main context 是 verbatim、external 是 extraction，落在三角中間。

三派的本質差異：

維度	Verbatim	Extraction	Compiled-Wiki
寫入時資訊損失	**無**（原文保存）	高（fact 抽完丟原文）	中（編譯到 wiki，但 raw sources 可保留）
查詢速度	中	**快**	中（markdown 直查）
跨 source 結構化	弱	中（KG）	**強**（cross-references）
對 LLM hallucination 的脆弱度	**低**	高	中
寫入成本	**零 LLM**	高	中

Verbatim 派的賭注：原文不丟，retrieval 找對片段就夠了，不需要 LLM 預先理解。像紙本檔案櫃。
Extraction 派的賭注：LLM 抽出來的 fact 就是「真實有用的資訊」，原文是雜訊。
Compiled-Wiki 派的賭注：真正有價值的不是 fact 或原文，是**人 + LLM 一起整理出的結構化知識**——這層結構是 wiki 的本體。

三派的選擇本質上是**對「知識本質」的不同假設**。Karpathy 寫 wiki gist 時，本質上在反對 extraction 派——他認為單純抽 fact 是不夠的，因為 fact 之間的關聯、上下文、層次，才是真正讓知識可用的東西。

§3.3 Beyond RAG ——Karpathy 為何說 RAG 不夠

「RAG is dead, long live the Wiki」——4 月以來這句話在 X / Hacker News 反覆被引用，源頭是 Karpathy 的 gist。但 RAG 真的死了嗎？

Karpathy 的論點：

> "the LLM is rediscovering knowledge from scratch on every question"

每次 query 都從 raw chunks 重新 synthesize 答案。這意味著：
1. **沒有複利**——你問過 100 次的東西，第 101 次仍從零開始

2. **沒有跨 query 一致性**——同樣問題不同時間問可能拿到不同答案
3. **沒有結構化沉澱**——零散 chunk 無法組成完整圖譜

Wiki 路線的解法：knowledge 編譯一次，cross-references 已就位，每次 query 直接讀已 synthesize 好的 markdown。

反對者的論點（社群中的 RAG 派）：

1. **Wiki 維護成本不可控**——LLM 自維護的可靠性無 benchmark，hallucination 會被「永久化」
2. **Wiki 無法處理快速變動的資料**——RAG 對 raw sources 直查，wiki 一旦過時就誤導
3. **Wiki 的 schema 是負擔**——你要先想好怎麼組織，這個前置成本對非典型場景很高
4. **DEV.to 的批評文證明**：在 client-side 環境（Claude Code、本地筆電）實作 wiki 的 background process 失敗率 ~50%

真實情況：兩條路解的不是同一個問題。

- **RAG 適合**：query 對 large dynamic corpus（文件多、更新頻、查詢面廣）
- **Wiki 適合**：personal / project knowledge（querie 集中、累積性強、需跨時間一致）

對個人工作流（我研究的這個主題、我寫的這個專案、我的個人偏好），Wiki 大幅優於 RAG。
對企業搜尋、客服 KB、即時資料查詢，RAG 仍是預設解。

Karpathy 的真實主張其實是：「**對個人 / agent 持續學習的場景**，當前社群押錯方向了，預設應該是 wiki 而非 RAG」——而不是「RAG 全面該死」。媒體標題化讓這個 nuance 流失了。

§ 3.4 中文 / 跨語言場景的限制

所有主流框架都是英文中心。中文場景有三個已知短板：

1. **Embedding 模型品質**——MemPalace 預設用 `all-MiniLM-L6-v2`（英文向）；Mem0 / Zep 預設 OpenAI embedding-3，中文 retrieval recall 比英文低 5-15 點（業界常見估計）
2. **Entity extraction 失誤率高**——Mem0 / Zep 從中文對話抽 entity / fact 時，人名地名混淆比英文高
3. **時序表達不規範**——「上週」「前年」「兩個月前」在 KG 抽取時容易標錯 validity window

當前緩解：
- 換中文向 embedding（如 BGE-large-zh、jina-embeddings-v3-zh）
- KG 抽取前用 LLM 把中文時序表達正規化成 ISO 8601 日期

但這層調校責任落在使用者，沒有任何框架預設處理好中文場景。**對台灣 / 中文市場開發 agent 的團隊，這是必須自己投資的工程**。

---

§ 4 Benchmark 解讀：別被數字騙了

§ 4.1 LongMemEval / LoCoMo 對照

框架	Benchmark	Model	Score	來源
Mem0 (vector)	LOCOMO	GPT-4o	66.9% LLM Score	Mem0 自家 paper
Mem0g (graph)	LOCOMO	GPT-4o	68.4%	Mem0 自家 paper
Mem0	LongMemEval	GPT-4o	**49.0%**	第三方（Atlan）
Zep	LongMemEval	GPT-4o	**63.8%**	Zep 公開
MemPalace	LongMemEval	mixed	96.6% R@5（爭議）/ 88.9% R@10（獨立重現）/ 60.3%（無 reranking）	MemPalace 自家
Letta	—	—	無公開	—
Cognee	—	—	無公開	—
LangMem	—	—	無公開	—
Supermemory	LongMemEval / LoCoMo / ConvoMem	—	自報領先	無第三方

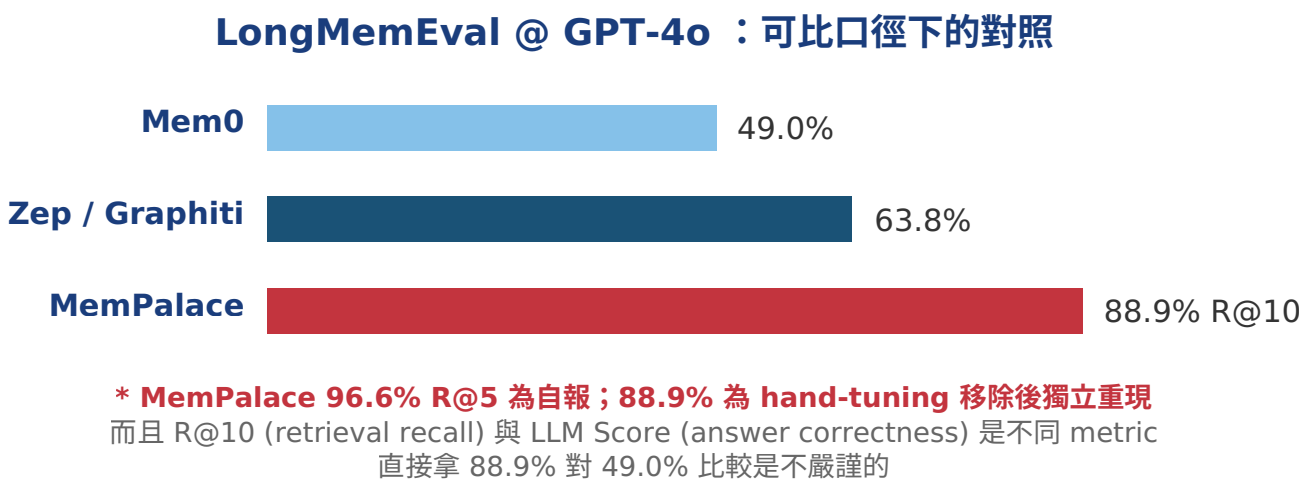

圖 6：LongMemEval 對照——Mem0 49% vs Zep 63.8% 是同口徑可比；MemPalace 88.9% 是不同 metric，須加註腳。

§ 4.2 為何 96.6% 不可直接拿來打 49.0%

四個口徑差異：

1. **Metric 不同**：MemPalace 量的是 **R@5（retrieval recall@top5）**——找到正確 chunk 的機率；Mem0 / Zep 量的是 **LLM Score（answer correctness）**——拿到 chunk 後最終答對的機率。一個是「找到」、一個是「答對」，找到不等於答對。
2. **Top-K gaming**：top_k=50 但 dataset 只有 19-32 items，等於「全部都拿出來」，retrieval 系統幾乎沒在過濾。
3. **Hand-tuning**：MemPalace 團隊在初版發布前已對 failing questions 做針對性修補。
4. **Model 不同**：Mem0 / Zep 都用 GPT-4o，MemPalace 用 mixed config。

真實可比的 number：MemPalace 的 60.3%（純 retrieval、no reranking、no LLM）vs Mem0 的 49.0%。仍領先約 11 點，但不再是 47 點的天文差距。

更深的教訓：2026 年 agent memory 的 benchmark 戰場已經 inflation。每家都在自家口徑下說自己最強。**買產品 / 選框架時，別看 marketing 數字，看 (a) 是否第三方獨立驗證、(b) 口徑是否一致、(c) 是否在你的場景重現**。

---

§ 5 認知架構視角

§ 5.1 Working / Episodic / Semantic / Procedural ——四元論

人腦記憶的心理學分類，當前 agent memory 的設計幾乎都在參考：

類型	人腦	Agent 對應	框架支援度
Working	短期、即時、容量小	Context window	全部都支援（即 LLM 本身）
Episodic	過去事件的回憶	對話歷史 / interaction log	Mem0 ✓ / Zep ✓ / Letta ✓ / MemPalace ✓ / Wiki 部分
Semantic	抽象事實、概念	結構化 fact / KG / 概念頁	Mem0 ✓ / Zep ✓ / Letta 部分 / MemPalace ⚠ / Wiki ✓
Procedural	怎麼做（技能、習慣）	system instruction / 工具調用 pattern	LangMem ✓ / Mem0 v1.0.0+ ✓ / Letta ✓ / 其他多缺

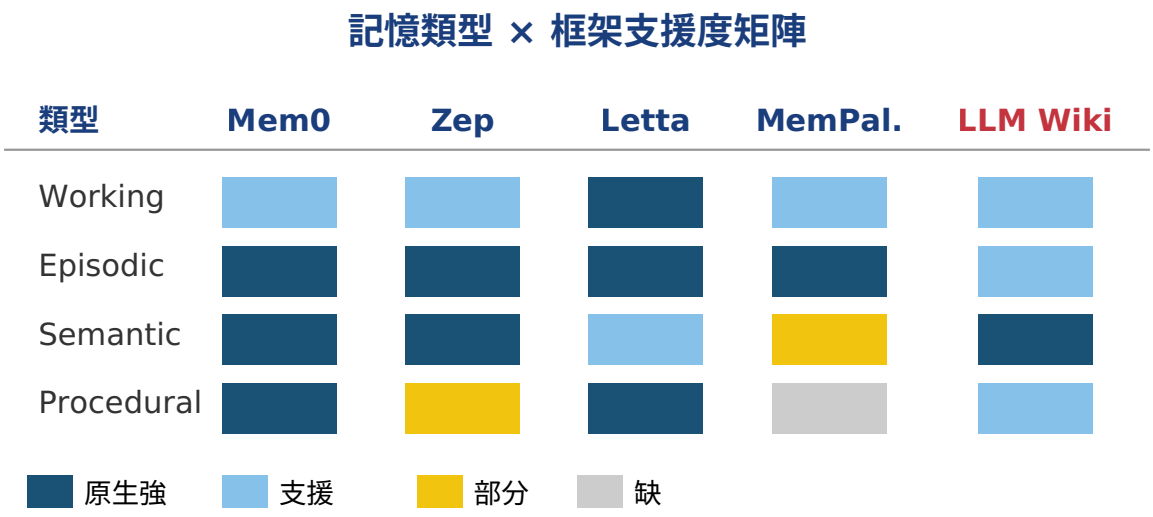

圖 7：記憶類型 × 框架支援度——四類記憶在五大框架的成熟度差異。MemPalace 弱在 procedural（無自更新行為）；LLM Wiki 弱在 episodic 自動化（要 daily log 編譯流程）。

詳細實作方式：

類型	Mem0	Zep	Letta	MemPalace	LLM Wiki
Working	LLM 內部	LLM 內部	Main context（OS RAM）	L0+L1（170 tokens）	注入的 wiki 段
Episodic	session 記錄	KG episodes	Conversation buffer	Hall (events)	Daily log
Semantic	extracted facts	KG nodes + validity	Memory blocks	Closet (compressed)	Wiki entity pages
Procedural	v1.0.0+ procedural	（弱）	Block update	（無）	schema (CLAUDE.md)

§ 5.2 System Prompt Learning ——Karpathy 提的第三範式

2025 年 5 月 Karpathy 在 X 丟下這段話：

> "We're missing (at least one) major paradigm for LLM learning. Not sure what to call it, possibly it has a name - system prompt learning? Pretraining is for knowledge. Finetuning (SL/RL) is for habitual behavior. Both of these involve a change in parameters but a lot of human..."

他在指當前 LLM 缺一個學習範式：

範式	機制	對應人類
Pretraining	改參數	從小到大累積知識（小學到大學）
Finetuning（SL/RL）	改參數	養成習慣 / 個性（職場磨練）
System Prompt Learning	**不改參數，改顯式規則 / 筆記 / 反思**	**寫筆記、總結教訓、列規則**（成人學習）

人類學新東西的主要方式不是「重訓大腦」，而是**寫筆記、列清單、跟自己對話**。當前 LLM 沒有這個機制——所有「我學到一個新教訓 → 把它寫成下次要遵守的規則」的路徑，都需要人類介入（手動修 prompt、手動 fine-tune）。

LLM Wiki 是 System Prompt Learning 的具體落地——agent 自己寫筆記、自己編譯結構化文件、下次 session 自己讀。Karpathy 把抽象的範式主張變成可執行的架構原型。

這也解釋了為什麼 2026 年所有 memory 框架感覺像「很複雜的 RAG」——它們都還沒跨過這條範式線。**真正的下一代不是「更好的 retrieve」，是 agent 學會自己寫 procedure**。Mem0 v1.0.0 加 procedural memory、LangMem 推自更新 system instruction，都是在這條線上的試探。

§5.3 Spreading Activation ——SYNAPSE 路線

人腦 retrieval 不是按 vector similarity，是按**聯想擴散**——你想到「咖啡」，腦中會自動激活「早晨」「義式」「下雨天」「困倦」等相關概念，激活強度隨關聯距離衰減。

ICLR 2026 的 SYNAPSE paper 把這個機制建模到 agent memory：

- 每個 memory node 有 activation level
- Query 觸發初始 activation
- Activation 沿 graph edges spread，按距離衰減
- 多個 query 的 activation 累加
- Top-k highest activation 作為 retrieval 結果

跟 vector similarity 的差異：
- Vector similarity：query 和 node 的「直接距離」
- Spreading activation：query 透過 graph 結構**間接**激活的所有 node

實際效果：對「上週我們討論的那個專案，後來怎麼樣了」這種多跳關聯的 query，spreading activation 顯著優於 vector similarity。但成本是 graph 結構必須夠豐富、edges 要夠準。**這也是為什麼 Zep 走 KG 路線在這類場景優於 Mem0 走 vector 路線**。

當前生產級框架幾乎都還沒原生支援 spreading activation。Zep 的 graph traversal 算近似版，但 activation level 的累加 / 衰減模型不完整。學術前沿與生產級的 12-18 個月 gap，這是其中一例。

§5.4 Episodic → Semantic → In-Weights ——三階段 Consolidation

人腦睡眠的核心功能之一是記憶 consolidation：白天的 episodic memory（具體事件）在睡眠中被 hippocampus 重播，逐步轉成 cortex 的 semantic memory（抽象規則 / 概念）。多年下來，最常用的 semantic memory 進一步「程序化」，變成不需思考的 implicit knowledge。

當前 agent memory 系統大多只 cover 第一段（episodic），少數 cover 到第二段（semantic）。**Karpathy 的真實野心是直通第三段**——把 wiki 烤回模型權重。

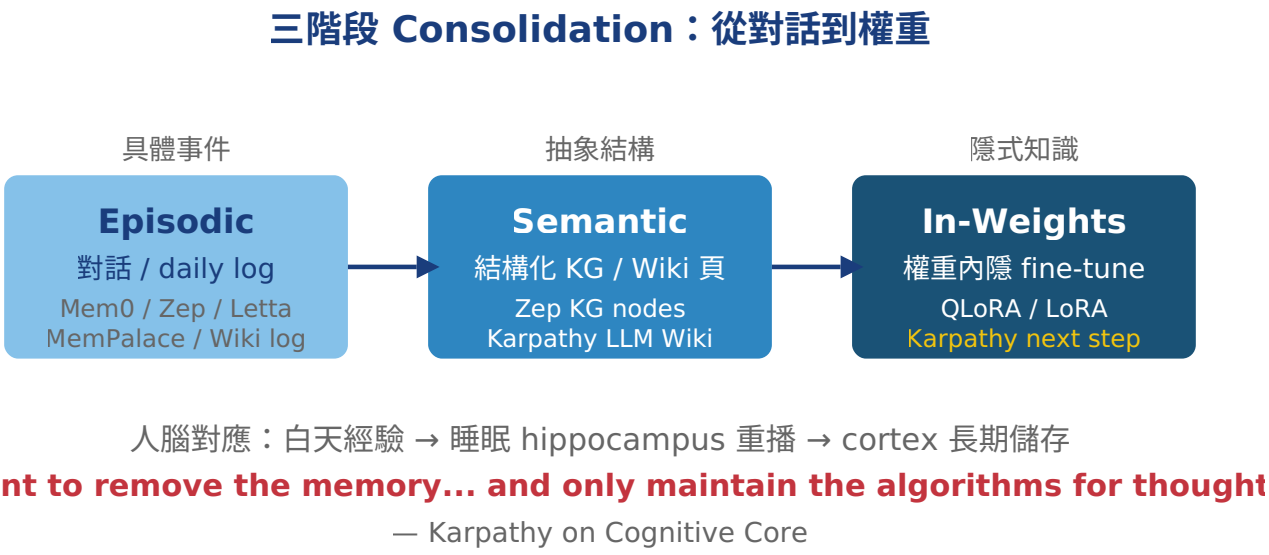

圖 8：三階段 consolidation——對應人腦的 episodic → semantic → in-weights 路徑。當前 agent memory 主要在第一、二階段，Karpathy 預告 next step 是把 wiki 烤回權重（第三階段）。

為什麼第三階段重要：

- **第一階段（episodic）**：每次都查 log，慢、貴、易雜訊
- **第二階段（semantic）**：查結構化文件，較快但仍需 context window 載入
- **第三階段（in-weights）**：知識直接內嵌權重，**零 retrieval cost、零 latency**，跟 pretraining 同等地位

但第三階段有兩個未解問題：
1. **Continual fine-tune 的 catastrophic forgetting**——學新東西會壞掉舊能力
2. **個人 vs 通用的權衡**——把使用者個人記憶烤進權重，等於每個人需要自己的 personal model

Karpathy 給的方向：Cognitive Core——瘦核心（~1B 參數）+ 外掛記憶 + 個人化 fine-tune。

> "I want to remove the memory... and only maintain the algorithms for thought"

如果這個願景成立，當前的 Mem0 / Zep / Letta 全部是過渡形態。它們在解第一、二階段，沒解第三階段。**最終的 agent memory 不是更好的外掛記憶服務，而是讓記憶部分變回權重的一環**。

§9.9 會深入這個推論。

---

§6 與 MCP / A2A 編排層的邊界

這份報告聚焦在「記憶層」，但 agent 系統還有「編排層」（orchestration）——多個 agent 怎麼協作、怎麼共享 context、怎麼分工。MCP（Model Context Protocol）和 A2A（Agent-to-Agent）是當前最受矚目的兩個編排協議。

三層堆疊：

層	代表	負責	不負責
Agent Capability	Claude Code、Cursor、Devin、各家 LLM	推理、tool use、執行	跨 session 持久 / 跨 agent 協作
Orchestration	MCP、A2A、LangGraph、AutoGen	多 agent 協作、shared context、tool 註冊	長期記憶、跨 session 知識累積
Memory	Mem0、Zep、Letta、MemPalace、LLM Wiki	跨 session 持久、知識累積、個人化	即時 tool 調度、agent 間訊息路由

三層**理論上獨立，實務上有重疊**：

- **Letta 越界進編排層**：原生支援多 agent 共享 memory blocks
- **MCP 越界進記憶層**：Anthropic Memory tool 是 MCP 上的 memory primitive
- **Karpathy LLM Wiki 同時在三層**：`schema.md` 是 capability 配置 + wiki 是 memory + agent 互讀 wiki 是編排

對選型的意義：

你的需求	落在哪層	對應產品
單 agent、跨 session 記住使用者偏好	Memory only	Mem0 / MemPalace
多 agent 共享 context	Memory + Orchestration	Letta + MCP
跨團隊 / 跨組織 agent 通訊	Orchestration only	A2A
個人 PKM、跨工具一致	Memory + 部分 Capability（schema）	LLM Wiki

為第二題（多 Agent 編排）預留：當你的需求是「多個 agent 各自跑、需要共享同一個記憶池」，問題就從「memory 框架選型」升級到「memory 該不該離開 agent 框架成為獨立 service」。Letta 的 memory block 是嘗試一，MCP-based shared memory 是嘗試二，但都還在早期。第二題會深入這個邊界。

---

§ 7 選型決策樹

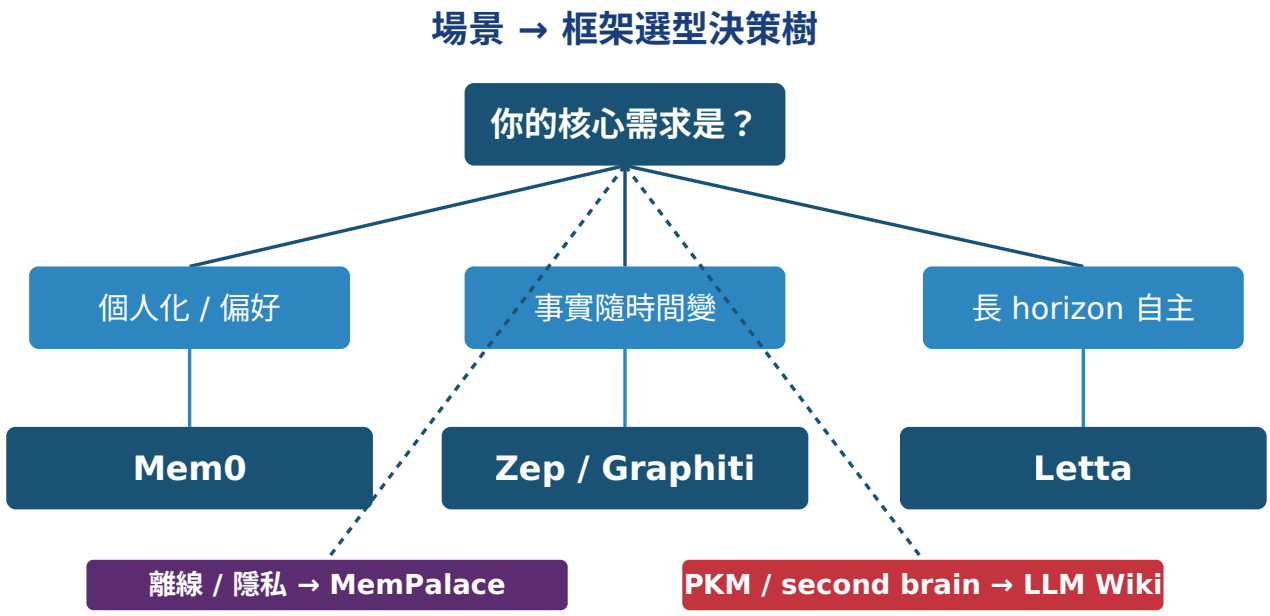

圖 9：選型決策樹——從核心需求快速定位到推薦框架。

場景對應速查：

場景	推薦	第二選擇	不推薦原因
個人化 chatbot / B2B copilot	**Mem0**	LangMem	Letta 太重；MemPalace 不適合 cloud
客服 KB（事實隨時間變）	**Zep / Graphiti**	Mem0g	Mem0 vector 不支援 temporal
跨任務累積技能的 autonomous agent	**Letta**	—	其他都假設 stateless
法律 / 金融知識追蹤	**Zep**	Cognee	需 validity window
完全離線、隱私敏感	**MemPalace** + **Cognee**	—	其他都依賴 cloud
個人 PKM / 開發者 second brain	**Karpathy LLM Wiki**	Obsidian + Mem0	沒有比 wiki 更適合 personal 累積
AI Coding Agent 跨 session 記憶	**LLM Wiki**（CLAUDE.md 模式）	Supermemory	RAG 對 code 場景太碎
多 agent 共享 context	**Letta** + MCP	Mem0 + 自建 sync	其他無原生支援
Air-gapped 企業	**Cognee** + Ollama	MemPalace	大廠 cloud 解不可用
已在 LangGraph 上的團隊	**LangMem**	—	換框架成本太高

---

§ 8 失敗模式與 anti-pattern

失敗模式	觸發場景	避免做法
Hallucination amplification	LLM extraction 寫入錯 fact，後續被多次強化	加 citation validator（rohitg00 v2 的補強）、定期 audit
Stale memory	Fact 演變但記憶系統未更新	用 Zep temporal model；或設 TTL
Cross-session contamination	多使用者共用 memory 但 scope 沒區分	嚴格區分 user_id / agent_id（Mem0 multi-scope）
Top-K gaming benchmark	為了好看的數字調 top_k 過大	評測時固定 top_k 為實際使用值
LLM Wiki client-side dropout	Background process 在 laptop 環境 timeout / silent fail	改 sync manual checkpoint（DEV.to 案例）
過度自動化掩蓋失敗	Karpathy 原 wiki 設計的 silent automation	「Simplicity First」——明確指令優於 silent automation
中文場景 embedding 退化	預設用英文 embedding 跑中文資料	換 BGE-zh / jina-zh
Schema 鎖定	LLM Wiki 一開始 schema 設定錯，後續難改	先做 minimal schema，邊用邊長
Fact 衝突無解	Mem0 / Zep 同主題抽出矛盾 fact	加 conflict resolution policy（v2 社群方案）

LLM Wiki 在 client-side 的具體實作陷阱（DEV.to 案例細節）：

7 個專案 3 週實測，「about half the time, the background process either timed out, failed to parse the transcript, or exited silently with no output」。

三大失敗：
1. Background process dropout
2. Transcript parser choking on long sessions
3. Silent pipeline failures（hash check 對但 pipeline 沒跑）

修正方向：
- Async background → **Sync within active session**
- Transcript parse → **File-based date filtering**
- Silent automation → **Manual checkpoint command**（如 `/close-day`）
- 檔放 `.claude/` → **搬到 project root**（`.claude/` 阻擋寫入）

> "Simplicity First. Minimum code that solves the problem"——引 Karpathy 自己的 principle 反向打他自己的初版設計。

---

§9 未來 12-24 個月應用想像

> 本節分兩層：(a) 已有雛形可指；(b) 純推測但有合理根據。

§9.1 Cognitive Core 願景 — (a) 雛形

> "I want to remove the memory... and only maintain the algorithms for thought" — Karpathy

雛形：
- Phi-3.5-mini（3.8B）+ 外掛 RAG 已在 edge 跑出近 GPT-3.5 表現
- Llama 3.2 1B 的小模型線正在強化「reasoning core，knowledge external」

12-24 個月推測：
- ~1B 參數的 cognitive core 模型線會獨立成型
- 配套 memory framework（Mem0 / Wiki）成為 edge agent 標配
- 對 on-device agent（手機、車載、機器人）打開市場

§9.2 Personal AI / Second Brain — (a) 雛形

雛形：
- Obsidian + Smart Connections（vector search 插件）
- Notion AI、Mem.ai
- Karpathy LLM Wiki + agent assistant（Claude Code 已支援 CLAUDE.md 模式）

推測：
- 2027 前出現「個人 AI 平台」整合 Obsidian / Roam + Mem0 + LLM Wiki
- 個人 Wiki 變成跨工具一致的 source of truth
- 跨應用 context 透過 personal wiki API 同步（例：你跟 ChatGPT 講過的內容，Claude Code 也能讀）

§9.3 AI Coding Agent 跨 session 記憶 — (a) 雛形

雛形（直接接續本專案 `reports/2026-04-claude-code/`）：
- Claude Code 的 `CLAUDE.md` 是手動版 LLM Wiki
- Cursor 的 `.cursorrules` 是手動版 procedural memory
- Supermemory + Claude Code 整合是自動版 Wiki

推測：
- IDE / agent 支援自動演化的 project knowledge wiki
- 跨專案 / 跨團隊的「coding pattern wiki」
- Pull request 時 agent 自動更新 architecture wiki
- 知識管理從「文件」升級到「agent 維護的活 KB」

§9.4 多 Agent 共享 Wiki — (b) 推測

> 為第二題（多 agent 編排研究）預留接口

目前缺：
- Mem0 / Zep / Letta 都假設單 agent / 單 user
- MCP shared context 還在早期
- Letta 的 memory blocks 是試水

推測架構：
- Wiki as service（HTTP API）
- 多 agent 透過 MCP read / write 同一 wiki
- Conflict resolution 透過 LLM 仲裁
- Audit trail 透過 git-tracked markdown

挑戰：concurrent write、stale read、agent 間信任邊界。

§9.5 教育 / Tutor agent — (b) 推測

架構：
- Wiki 記錄學生薄弱概念、學習偏好、進度
- 跨學期持續追蹤
- agent 自動調整教法（從 episodic learning event 反推 procedural teaching pattern）

雛形：Khanmigo（Khan Academy AI Tutor）已在試 cross-session memory，但仍弱在 procedural（記住「這個學生對視覺解釋反應好」）。

§9.6 企業 KM 取代 Confluence — (b) 推測

目前痛點：
- Confluence / Notion 內容不會自更新
- 員工不維護 → 文件腐化
- 搜尋體驗差

推測：
- Wiki maintained by agent，從 Slack / Email / 會議錄音 / git commit 自動更新
- 員工只負責「讀」+「修正錯誤」
- agent 每週 audit 內容一致性

挑戰：權限管理、隱私邊界、誤更新的回滾。

§9.7 Embodied / Robotics — (b) 推測

挑戰：
- 機器人需「跨任務技能累積」（學會開門 → 學會其他類似動作）
- 物理世界的 episodic memory 比文字密集得多
- Procedural memory（運動技能）難以用 markdown 表達

推測：
- Robotics-specific memory system（含 trajectory、視覺場景、力覺）
- 與 LLM-based memory 雙層架構：高層 LLM Wiki 記任務語義，低層 embedding 記 motor pattern

§ 9.8 科研 Autoresearch — (a) 雛形

雛形：karpathy/autoresearch——AI agents 自動跑單 GPU nanochat 訓練 + 實驗記錄。

推測：
- 跨實驗 wiki：記錄 ablation、超參、結果、假設修訂
- agent 自動 propose 下個實驗
- 文獻 review wiki + 實驗 wiki 雙向 cross-reference

對比現況：當前 ML 研究員手動維護 Notion / Notebook，效率瓶頸大。

§ 9.9 Wiki → Fine-tune Consolidation Loop — (b) 高賭注推測

> Karpathy 預告的 next step

架構：
1. Daily 對話 + raw sources
2. LLM 編譯成 wiki（episodic → semantic）
3. 累積一定量後，**用 wiki 內容做 personal fine-tune（QLoRA / LoRA）**
4. Fine-tune 完，原 wiki 部分內容可以從 prompt 中移除（已內隱權重）
5. 持續循環

這對應人腦的 sleep consolidation：白天經驗（episodic）→ 睡眠 hippocampus 重播 → cortex 長期儲存（semantic / implicit）。

為何是高賭注：
1. **Continual fine-tune 的 catastrophic forgetting 仍未解** — 學新東西可能壞掉舊能力
2. **Personal model 的經濟學**：每個人需要自己的 fine-tune cycle，cost 不低
3. **隱私**：fine-tuned weights 含個人記憶，等於數位人格的物質載體

潛在贏家：
- 個人化 fine-tune as a service（類比 1990s 個人電腦概念）
- "Personal Foundation Model"——你的個人 1B 模型，跟你一起學一輩子

風險：
- 模型能力不夠強，fine-tune 也撈不出來
- 巨頭壟斷（OpenAI / Anthropic 不釋放 weights）
- 隱私 / 安全災難（你的個人 weights 被偷 = 你的數位人格被偷）

如果這條路走通，當前 Mem0 / Zep / Letta / MemPalace 全部是過渡產品。**真正的 agent memory 不是更好的外掛，是讓記憶部分回到模型本身**。

---

§ 10 能力限制聲明

本研究存在以下無法克服的限制，撰寫時已盡量誠實標註：

1. **無法實測各框架的 latency / API 成本**——所有性能數字引自官方 / 第三方公開資料，我們未自跑
2. **Karpathy 觀點為個人視角**——LLM Wiki gist 是非正式 GitHub gist 而非 peer-reviewed paper，社群仍在實證；System Prompt Learning 是 X 推文非完整論述；Cognitive Core 是 Dwarkesh podcast 訪談摘要。**不應視為已驗證的工程藍圖**
3. **LLM Wiki 自維護的可靠性數據缺乏**——目前僅有少數個案報告（包含 DEV.to 的批評文），無系統性 benchmark
4. **Benchmark 數字可比性差**——Mem0 / Zep / MemPalace 各家用不同 model + embedding + dataset 口徑，跨家直接比是不嚴謹的；本報告 § 4 已盡量拆口徑，但仍可能誤導
5. **MemPalace 才剛釋出（2026-04-08）**——社群評測還在劇烈震盪，本報告引用的 88.9% 獨立重現是當前最可信數字，但未來 1-2 個月內可能再有大幅修正
6. **中文場景能力**——所有主流框架都是英文中心，中文場景的真實表現本研究未實測，僅引用業界常見估計
7. **學術原型（HiMem / SYNAPSE / TiMem / MAGMA）的工程落地未驗證**——這些 paper 給出概念貢獻但未必有可用 codebase，與生產級框架直接比較是不公平的
8. **§ 9 應用想像分 (a)(b) 兩層**：(a) 已有雛形可指、(b) 純推測但有合理根據；(b) 部分屬於趨勢判斷而非結論
9. **資料截止日 2026-04-29**——agent memory 領域 2026 年迭代極快，本報告 1-2 個月後可能部分過時，特別是 MemPalace、LLM Wiki 社群、Letta MemFS 的新進展

主管 / 採購 / 投資判斷請以本聲明為前提，不要把報告當作「最終答案」。**這個領域當前最好的姿態是「持續關注 + 小型試點」**，而不是「選定一家 all-in」。

---

§ 11 參考資料

完整 URL 與訪問日期列在同層 `sources.md`。本節給出主要分類索引。

Karpathy 一手來源

- LLM Wiki gist（karpathy/llm-wiki，2026-04-04）
- System Prompt Learning X 推文（status 1921368644069765486，2025-05）
- Dwarkesh Patel podcast 訪談（2025-10）
- Latent.space S3 演講「Software 3.0」
- karpathy/autoresearch GitHub repo

框架官方資源

- Mem0 docs + State of AI Agent Memory 2026 blog
- Zep Knowledge Graph product page + Graphiti GitHub
- Letta docs + GitHub
- MemPalace GitHub（milla-jovovich/mempalace）+ 官方介紹
- Cognee docs

第三方對比

- Atlan：Best AI Agent Memory Frameworks 2026
- Letta forum：Letta vs Mem0 vs Zep vs Cognee
- DEV.to：Top 6 AI Agent Memory Frameworks for Devs 2026
- Graphlit：Survey of AI Agent Memory Frameworks

學術論文

- MemGPT（arXiv:2310.08560，2023-10）
- Mem0 LOCOMO（arXiv:2504.19413，ECAI 2025）
- ICLR 2026 MemAgents Workshop（OpenReview U51WxL382H）
- HiMem / SYNAPSE / TiMem / MAGMA / GAM / A-MEM / MIRIX 各論文

批評 / 修訂

- Nicholas Rhodes：MemPalace Review（2026-04）
- DEV.to：I Over-Engineered Karpathy's Agent Memory（2026-04-17）
- rohitg00：LLM Wiki v2 gist
- arXiv 2604.21284：Spatial Metaphors for LLM Memory（MemPalace 學術批評）

產業實踐

- LinkedIn Cognitive Memory Agent（ZenML LLMOps Database）
- VentureBeat：Karpathy LLM Knowledge Base 報導
- Memco / Gamgee / Epsilla / TecAdRise / MindStudio 商業化解讀

報告完。資料截止 2026-04-29。下一份相關研究：多 Agent 編排協議（MCP / A2A）與 shared memory 邊界。